What Transfers from Text-to-SQL to Text-to-Cypher? A Controlled Study of Reasoning, Selection, and Training Recipe

Cyrus Parsons

Abstract

Text-to-SQL has produced a large toolbox of techniques—chain-of-thought (CoT) distillation, self-consistency, execution-based selection, reinforcement learning—but it is unclear which of them transfer to the structurally different task of Text-to-Cypher. We run a controlled, matched-pipeline study to find out, unified by a single *constrained-output-space hypothesis*: a Cypher graph pattern is a single *connected* object with few semantically-equivalent surface forms, whereas SQL is *compositional* with many. This predicts that (P1) string-diversity methods fail, (P2) execution-grounded methods still transfer, and (P3) CoT/decomposition hurts. Our central result is negative and, to our knowledge, the first controlled demonstration of it: CoT distillation—the highest-profile SQL transfer—does *not* improve Text-to-Cypher and modestly hurts it. The effect holds across two model families (Gemma-2-9B, Llama-3.1-8B), naive and execution-verified traces, and in-distribution, compositional, and length-generalization regimes, with paired-bootstrap confidence intervals excluding zero throughout. Apparent gains reported under an earlier framing are attributable to a stronger supervised fine-tuning recipe and a leaked benchmark split, not to reasoning. We give a causal mechanism: CoT's decompositional bias fragments connected graph patterns, and a holistic-reasoning ablation recovers ∼40% of the penalty by cutting fragmentation 5–6×. Consistent with the hypothesis, execution-grounded selection transfers (+1.1pp exact match) while string self-consistency does not. We close with a methodological lesson: a published artifact used as a control, rather than a matched in-pipeline baseline, produced months of false "CoT helps" conclusions.

1 Introduction

Text-to-SQL is a mature field with a well-stocked toolbox: CoT distillation [He et al., 2025, Tai et al., 2023], self-consistency, execution-based candidate selection, and reinforcement learning have each produced large gains. Text-to-Cypher—translating natural language to the graph query language of Neo4j [Ozsoy et al., 2025]—is younger and structurally different, and it is tempting to assume the SQL toolbox carries over. This paper asks, in a controlled setting, *which techniques actually transfer, and why.*

Our organizing idea is the **constrained-output-space hypothesis** (§3): a Cypher basic graph pattern is a single *connected* object admitting few semantically-equivalent surface forms, unlike SQL, which composes from sub-queries and joins and admits many. This structural difference yields three testable predictions about transfer, P1–P3, developed in §3.

Our headline finding tests the most important transfer—CoT distillation—and it is negative. Under a matched pipeline where the *only* variable is the training target (a reasoning trace followed by the query, versus the query alone), CoT distillation modestly *degrades* Text-to-Cypher across two model families and every regime we test. We trace an earlier positive framing of this same project to two confounds: an unmatched baseline (a published adapter used as a control) and a leaked cross-split evaluation. We then explain the negative result mechanistically and show which techniques *do* transfer.

Contributions.

1. **A controlled matched-baseline result** that CoT distillation does not help, and modestly hurts, Text-to-Cypher—across Gemma-2-9B and Llama-3.1-8B, naive and execution-verified traces, and in-distribution / compositional / length-generalization regimes (§5.1).
2. **A causal mechanism**: decomposition fragments connected graph patterns; a holistic-reasoning ablation recovers ∼40% of the penalty (§6).
3. **A positive transfer**: execution-grounded (MBR) selection beats string voting while string self-consistency fails, confirming the hypothesis (§5.4).
4. **A methodological lesson**: always train a matched in-pipeline baseline before attributing a delta to an intervention (§7).

2 Related Work

2.1 The Text-to-SQL toolbox

The techniques whose transfer we test were each developed and validated on Text-to-SQL.

Reasoning and decomposition. Chain-of-thought prompting [Wei et al., 2022] generates intermediate reasoning before the answer. For Text-to-SQL, Tai et al. [2023] systematically compared CoT strategies and found QDecomp+InterCOL—question decomposition with intermediate schema-element identification—most effective (+5.2pp on Spider dev with Codex). *Distilling* such reasoning into small models is the highest-profile transfer candidate: STaR-SQL [He et al., 2025] reports 86.6% Spider execution accuracy from an iterative rationale-bootstrapping loop, and Rossiello et al. [2025] distill CoT rationales for Text-to-SQL directly. Two details of the SQL evidence matter for what follows. First, the clean same-data comparison inside STaR-SQL (CoT-SFT vs. direct-SFT) is +6.4pp; the celebrated +18pp includes a trained verifier doing best-of-16 selection. Second, several strong results bundle CoT with other machinery: Struct-SQL [Thaker and Bresler, 2025] structures the reasoning around execution plans, ExCoT [Yao et al., 2025b] adds iterative DPO, and Liu et al. [2025] show DPO *requires* CoT to work for Text-to-SQL. Decomposed pipelines (DIN-SQL Pourreza and Rafiei 2023; DTS-SQL Pourreza and Rafiei 2024; CHASE-SQL Pourreza et al. 2025a) make the same bet at inference time. The theoretical basis for decomposition is QDMR [Wolfson et al., 2020, 2022].

Sampling, selection, and RL. Self-consistency [Wang et al., 2022] samples multiple reasoning paths and majority-votes the answer; execution-grounded variants cluster candidates by their *result sets* rather than their strings. Agentic refinement loops (MAC-SQL Wang et al. 2025) add error-driven correction. GRPO [Shao et al., 2024] is the dominant RL algorithm for query generation: Arctic-Text2SQL-R1 [Yao et al., 2025a] reaches #1 on BIRD with execution-only rewards, and Reasoning-SQL [Pourreza et al., 2025b] uses composite partial rewards.

2.2 Text-to-Cypher

Compared to Text-to-SQL [Mohammadjafari et al., 2024, Liu et al., 2024], Text-to-Cypher is younger. Early systems were rule- or template-based [Kobeissi et al., 2021, Nie et al., 2022]. The Neo4j Text2Cypher benchmark [Ozsoy et al., 2025]—44,387 question–schema–Cypher triplets over

966 schemas—established the standard evaluation: fine-tuned GPT-4o leads at 0.8017 GLEU, and the benchmark's fine-tuned Gemma-2-9B baseline reports 0.5560. Follow-up work explores schema filtering [Ozsoy, 2025c], data pruning [Ozsoy, 2025b], iterative refinement [Ozsoy, 2025a], and multilingual adapters [Ozsoy, 2026]. Auto-Cypher [Tiwari et al., 2025] generates fully-executable synthetic training data (using CoT during *generation*, not at inference); Yang et al. [2026] combine synthetic data, preference learning, and schema retrieval; Hornsteiner et al. [2024] build a live-schema pipeline with error-correction loops; Feng et al. [2023] chain prompts for a Chinese KBQA task. Closest to our question, Tran et al. [2025] apply GRPO with structured-extraction rewards to Qwen2.5-3B (GLEU 0.7701 on Neo4j 2024)—the first RL for Text-to-Cypher—and STRuCT-LLM [STRuCT-LLM Authors, 2025] trains jointly on SQL and Cypher with topology-aware rewards. Benchmarks beyond Neo4j's include CypherBench [CypherBench Authors, 2025], ZOGRASCOPE [ZOGRASCOPE Authors, 2025]—a manually annotated Pole-graph benchmark with IID, compositional, and length-generalization splits—and Mind the Query [Chauhan et al., 2025].

2.3 Positioning

Prior Cypher work *imports* individual SQL techniques (RL, refinement loops, schema filtering) and reports absolute numbers, implicitly assuming the toolbox transfers. None runs matched-pipeline controls in which a single technique is the only variable, and the highest-profile transfer of all—CoT distillation, assumed positive on the strength of the SQL literature—has not been tested against a matched direct-answer baseline at all. That is the gap this paper fills; our matched controls overturn the assumed-positive for CoT while confirming transfer for execution-grounded selection.

3 The Constrained-Output-Space Hypothesis

The structural difference between the two query languages can be stated precisely. A SQL query *composes*: sub-queries, CTEs, and join clauses are separable units that can be drafted independently and then combined, and the same semantics admits many surface realizations—`JOIN` vs. correlated subquery, alias renaming, clause reordering, `IN` vs. `EXISTS`. A Cypher query is dominated by its basic graph pattern, e.g. `(a)-[:R]->(b)-[:S]->(c)`, which is a single *connected* object: every node variable is shared between adjacent relationship atoms, so no sub-part can be produced independently without violating a global consistency constraint. The pattern must be assembled as a whole, and—for a given schema and question—it typically has one canonical surface form (Neo4j's own training data reinforces this canonicality).

Write $\mathcal{Y}(q)$ for the set of correct surface forms for question q. The hypothesis is twofold: $|\mathcal{Y}_{\text{Cypher}}(q)| \ll |\mathcal{Y}_{\text{SQL}}(q)|$, and correct Cypher outputs are *globally coupled* (connected patterns) where SQL outputs are *locally decomposable*. Three predictions about the SQL toolbox follow:

- **P1 (string diversity fails).** With essentially one canonical form, sampling produces little useful diversity, so string-vote self-consistency cannot help.
- **P2 (execution grounding transfers).** Methods that operate on *results* bypass the surface-form bottleneck and still help.
- **P3 (decomposition hurts).** Decomposing a connected pattern fragments it; the right inductive bias for SQL is the wrong one for Cypher.

A corollary sharpens P3. When the benchmark supplies entity values explicitly (as ZOGRASCOPE does), query generation is largely a *copy-and-place* task: the values are given, and the work

is placing them in the right structural slots of one connected pattern. Direct-answer generation copies each value once. Reasoning-first generation re-states values, paths, and structure in prose before the query, and every re-statement is an opportunity to corrupt, drop, or add—so the hypothesis predicts not just that decomposition mis-assembles structure but that longer reasoning chains *accumulate* errors on such tasks. Both effects appear in §6.

4 Matched Experimental Protocol

Every comparison in this paper fixes the base model, the QLoRA configuration (4-bit NF4, r=64, α=64, all-linear targets, `paged_adamw_8bit`), the completion-only loss masking, and the CoT-distillation procedure (traces from GPT-oss-120B), and varies *only* the training target: the reference query alone (*direct*) versus a reasoning trace followed by the reference query (CoT). Both arms train on the identical set of instances (those with a valid trace), so no data differs between them. We instantiate this protocol on Cypher (Neo4j 2024; ZOGRASCOPE), SPARQL (LC-QuAD 2.0), and SQL (§5.3).

Trace generation. Reasoning traces are distilled from GPT-oss-120B (a 117B-parameter open-weight MoE reasoning model). For the primary Neo4j arm we generated traces for all 39,554 training examples (99.997% success; ∼$50). Each trace follows a four-step QDecomp+InterCOL decomposition adapted from SQL [Tai et al., 2023] to graphs: (1) decompose the question into sub-questions; (2) link each to schema elements (node labels, relationship types, properties); (3) identify the traversal pattern; (4) construct the query clause by clause. The reference query is shown to the teacher, which reasons backward from it (rationalization); the training target always uses the original reference query, never the teacher's re-generation. A separate *forward* variant for the verified-trace experiment (§5.2) hides the reference, samples k=4 candidate rationale–query pairs, and keeps only traces whose query *executes to the reference result* (the STaR-SQL ingredient He et al. 2025); the keep rate is 57.6%, comparable to STaR-SQL's ∼50% on SQL.

Training. We fine-tune `google/gemma-2-9b-it` (and `Llama-3.1-8B-Instruct` as the second family) with the QLoRA configuration of the Neo4j baseline [Ozsoy et al., 2025, Dettmers et al., 2023, Hu et al., 2021]: 4-bit NF4 with double quantization and BF16 compute, LoRA r=64, α=64, dropout 0.05 on all linear projections, 1 epoch, learning rate 2×10^{-5}, effective batch 32, max sequence length 1,600. Loss is computed over response tokens only (completion-only masking); prompt tokens are masked to the cross-entropy ignore index. The direct arm's target is the query y; the CoT arm's target is $r \oplus y$. This masking choice is itself a measured variable in §5.4.

Inference and metrics. Greedy decoding under the training-matched prompt for each arm (the CoT arm's prompt requests step-by-step reasoning; the query is parsed after a "Cypher output:" marker). *Translation-based:* GLEU [Wu et al., 2016] via HuggingFace Evaluate and whitespace-normalized string exact match (EM), on the full Neo4j test set (n=4,833). *Execution-based:* predicted and reference queries are executed—against the public Neo4j demo databases for the Neo4j benchmark (n=2,471 DB-eligible instances) or a local Pole-graph container for ZOGRASCOPE—and lexicographically sorted result sets are compared for exact match. For SQL we additionally report a *canonicalized* EM (sqlglot AST normalization), which credits case, whitespace, quoting, and structural normalization. *Uncertainty:* for the central deltas we report paired bootstrap 95% confidence intervals (B=10,000 resamples over instances; for corpus GLEU we resample per-instance

Table 1: Matched direct-vs-CoT on Neo4j 2024 (our pipeline; only the training target differs). CoT hurts both families. Brackets: paired bootstrap 95% CIs on the effect; all exclude zero.

Model	Target	GLEU	String EM	Exec EM
Gemma-2-9B	direct	**0.7854**	**0.4331**	**0.2975**
Gemma-2-9B	CoT	0.7682	0.3799	0.2554
Llama-3.1-8B	direct	**0.7680**	**0.4223**	**0.2865**
Llama-3.1-8B	CoT	0.7416	0.3000	0.2161
CoT effect (Gemma)		−0.0172 [−0.025, −0.010]	−0.0532 [−0.063, −0.044]	−0.0421 [−0.060, −0.025]
CoT effect (Llama)		−0.0264	−0.1223	−0.0704 [−0.087, −0.054]

CIs for the Llama translation deltas are omitted (matched-prompt prediction files pending retrieval from the cluster); the point estimates are 1.5–2.3× the Gemma effects, whose CIs exclude zero.

Table 2: Decomposition of the previously reported "+0.1227 GLEU from CoT" on Neo4j 2024.

Component	Δ GLEU
Pipeline (Neo4j published adapter → our direct-answer adapter, same inference)	+0.1399
CoT (our direct-answer → our CoT)	−0.0172
Net (previously reported as "CoT")	+0.1227

sufficient statistics, for the exact-match metrics per-instance outcomes). All reported intervals exclude zero.

Leakage discipline. After discovering that a merged-split ZOGRASCOPE setup contained 58.6% instance overlap (invalidating an earlier "length-generalization SOTA" claim, §5.2), every train/test pairing in this paper is verified for overlap at the question level *and* the (question, query)-pair level, not merely by instance ID; ID-disjointness alone misses the Neo4j benchmark's 31.7% verbatim question leakage.

5 Results

5.1 The matched negative result on Text-to-Cypher

Table 1 reports the matched comparison on the Neo4j 2024 benchmark. For *both* model families, CoT *lowers* every translation and execution metric. A secondary claim from the earlier framing also fails: CoT does not improve syntactic validity. The direct-answer Gemma produces *fewer* invalid queries than its CoT counterpart (100 vs. 114 execution errors); the earlier "CoT halves prediction errors" observation compared against the unmatched published adapter's 242.

The positive result reported under an earlier framing of this project decomposes cleanly (Table 2): the apparent "+0.1227 GLEU from CoT" was a comparison against a *published* adapter run through our inference harness, not a matched control. Replacing that baseline with our own direct-answer model shows the gain was the training pipeline (+0.1399); CoT itself *subtracts* (−0.0172).

Table 3: Clean ZOGRASCOPE execution accuracy (per-split, leakage-verified). Direct wins every split; paired bootstrap 95% CIs on the difference all exclude zero.

Split	n	Direct	CoT	Δ (direct − CoT) [95% CI]
IID	768	0.8255	0.6094	+0.216 [+0.184, +0.249]
Compositional	1349	0.6612	0.4181	+0.243 [+0.214, +0.273]
Length (dist. shift)	1253	0.4397	0.2306	+0.209 [+0.177, +0.241]

5.2 Robustness: leakage, verified traces, and generalization splits

Not a leakage artifact. Neo4j 2024 contains 31.7% of test questions verbatim in training, but the leakage is adversarial (often a *different* gold query); on genuinely unseen questions the direct-answer model beats CoT by +0.068 EM, so CoT loses *most* where leakage is absent. **Verified traces don't rescue it.** A natural objection is that our traces are post-hoc rationalizations rather than verified reasoning. Retraining both arms on the same 3,938 execution-verified forward-generated instances (the STaR-SQL ingredient; keep rate 57.6%, comparable to STaR-SQL's on SQL—so forward Cypher generation is not intrinsically harder) still yields CoT below direct (exec EM 0.0939 vs. 0.1052, with 60% more invalid queries); absolute numbers are low because the training set is small, but the comparison is internally matched. **Generalization.** On clean, per-split ZOGRASCOPE (leakage-verified), direct-answer wins every split including length (Table 3)—overturning an earlier "length-generalization SOTA" claim that was a cross-split leakage artifact.

5.3 Cross-formalism

Table 4 extends the matched comparison to SPARQL and SQL. CoT hurts SPARQL (LC-QuAD 2.0 over Wikidata; both arms face the same entity-linking difficulty, so the delta is internally valid) as it does Cypher. On synthetic SQL, CoT hurts under string exact match—but our own hypothesis (P1 inverted) warns that string metrics undercount SQL correctness, since SQL admits many equivalent surface forms. We therefore re-scored the SQL predictions at three levels of increasing semantic awareness: raw string EM (−0.0473), normalized EM (−0.0485), and sqlglot canonical-AST EM (−0.0477; paired bootstrap 95% CI [−0.056, −0.039], n=5,851). *The deficit does not close as the metric becomes more semantic*—the three deltas agree to within 0.001—so surface form is not inflating the gap. Per-complexity, CoT is worse or tied in every stratum, including the compositional ones (subqueries −0.041; CTEs and multi-join near floor for both arms). Canonical-AST EM remains a lower bound on semantic equivalence (it does not credit alias renames or `JOIN`↔subquery rewrites), so the decisive test is *execution accuracy on Spider*, which is staged and pending; we deliberately draw no conclusion here about whether CoT helps SQL in this pipeline. [TODO: Fill the Spider execution row when the run lands. Per advisor guidance, do not draw the compositional-prior conclusion from this table until that number exists.]

5.4 What does transfer: execution-grounded selection and a stronger recipe

Selection (P1, P2). Consistent with P1, string-vote self-consistency at five samples falls *below* single-sample greedy decoding (0.2509 vs. 0.2554 exec EM): Cypher's near-canonical output space leaves nothing to vote across, so sampling only adds noise. Consistent with P2, clustering the same five candidates by their *execution results* (MBR selection) rises above both (0.2665; Table 5): semantically-equivalent-but-textually-different candidates collapse into one result cluster and reinforce each other. The oracle ceiling (any of the five candidates correct) is 0.4302. The evaluated

Table 4: Matched CoT effect across formalisms (our pipeline). Negative everywhere measured; the decisive SQL execution-accuracy control is pending.

Formalism (dataset)	Metric	CoT effect
Cypher (Neo4j 2024)	GLEU	−0.0172
SPARQL (LC-QuAD 2.0)	GLEU	−0.1470
SQL (synthetic)	canon. EM	−0.0477
SQL (Spider)	execution acc.	*pending*

Table 5: Self-consistency selection on the CoT model (Neo4j exec subset). Execution-result voting transfers; string voting does not.

Selection method	Exec EM
String-vote SC@5	0.2509
Greedy (single)	0.2554
Execution-vote SC@5 (MBR)	**0.2665**
Oracle (any of 5 correct)	0.4302

instances partition three ways: 57% are generation-bound (no correct candidate among the five—only more sampling or a stronger base model can help), 27% are solved by MBR's result-cluster vote, and 16% are verifier-addressable (a correct candidate exists but loses the vote; in 77% of these it is the lone correct candidate). A trained verifier in the STaR-SQL/ORM style is thus the natural future extension, with its expected gain bounded by that 16%.

Training recipe. The second surviving positive is mundane but large: our direct-answer model exceeds the published fine-tuned baseline by +0.23 GLEU (0.7854 vs. 0.5560). This decomposes into +0.09 of *evaluation harness* (running the published adapter through our own inference—full-schema prompts rather than 1,600-token truncation—yields 0.6455; GLEU is also highly sensitive to tokenization for punctuation-dense Cypher) and +0.14 of *training*. The configurations differ in one decisive place: the published recipe trains with full-sequence loss over schema-heavy prompts (with a ∼30:1 schema-to-answer token ratio, ∼97% of the gradient reconstructs schemas), while ours masks the prompt and spends the entire loss on the query. A controlled full-sequence ablation of our own pipeline attributes +0.044 GLEU to the masking choice alone (0.7854 completion-only vs. 0.7415 full-sequence); the remainder of the training gap is unexplained (sequence packing is the leading suspect) and we flag it as such. The practical point stands independently: completion-only masking on long-schema inputs is a cheap, reusable recipe improvement, and the model it produces is—to our knowledge—the strongest open-weight result on this benchmark under our harness, with the important caveat that cross-harness absolute comparisons are unreliable (which is itself one of this paper's lessons).

6 Mechanism: why CoT hurts Cypher

6.1 The failure is compositional, not relational

On clean ZOGRASCOPE with execution ground truth, we analyze the 956 instances where the direct model is execution-correct and the CoT model is not. Only 3% of these failures use rela-

Table 6: Reasoning-format ablation (ZOGRASCOPE, execution accuracy, common instances). Holistic framing recovers part of the CoT penalty; no format closes the gap to direct.

Config	Length	IID	Comp.
Direct-answer	**0.475**	**0.826**	**0.661**
QDecomp-CoT	0.258	0.609	0.418
Holistic-CoT	0.303	0.710	0.511
Enum-CoT (E4)	0.231	0.669	0.429

tionships unrelated to the reference—*97% identify the correct relationships*. Schema linking (the InterCOL step) works. The failure is in assembly: 49% use the right relationships in the wrong connected structure (wrong order, misplaced filters, or fragmentation of one connected traversal into disconnected `MATCH` clauses), 32% drop hops, 5% add hops. The model has the right pieces and puts them together wrong—precisely what P3 predicts for a globally-coupled output.

6.2 Causal test: decomposition drives fragmentation

If decomposition is the assembly-breaker, a reasoning format that keeps the reasoning but removes the decomposition should recover part of the penalty. We trained a *holistic-path* CoT variant whose trace states the full connected path explicitly, with no sub-question decomposition, and an *enumeration* variant (E4) that additionally lists and counts every required relationship before constructing. Table 6 reports the ladder. Holistic-CoT beats QDecomp-CoT on every split (+4.6 to +10.0pp), recovering ∼40% of the CoT penalty, and the gain is localized: fragmentation of connected paths drops from 11% to 2% (length split) and 6% to 1% (regular)—a 5–6× reduction. Decomposition is thus *causally* a major component of the harm. Two boundaries: no reasoning format closes the gap to direct-answer (holistic still trails by 12–17pp), and *more* reasoning structure backfires—E4's enumeration scaffolding underperforms holistic everywhere and partially reintroduces fragmentation. Minimal reasoning is the best reasoning; none is better still.

6.3 The residual: error accumulation on a copy-and-place task

What explains the rest of the penalty? Not path truncation: although the reasoning does commit to too-short paths on long traversals (and does so at the planning stage—in 99–100% of dropped-hop failures the hop is already missing from the trace), truncation *alone* accounts for only 2% of holistic-CoT failures, and E4 behaviorally confirmed its insufficiency by fixing short-path hop deficits without improving accuracy. The robust CoT-specific signature is *filter-value corruption*: the CoT model filters on wrong values 3.7–5× more often than the direct model (16% vs. 4% of length-split predictions), and in 92% of these cases the wrong value already appears in the reasoning trace—corrupted during reasoning, then faithfully copied into the query (e.g., "vehicle-related crimes in BL5" acquiring a spurious `"vehicle"` filter). ZOGRASCOPE hands entity values to the model, making generation a copy-and-place task (§3): the direct model copies each value once; the CoT model re-states values, paths, and structure in prose first, and each re-statement risks corruption. This error-accumulation account unifies the ladder's monotonicity—more reasoning is strictly worse—though we note its bounds honestly: filter corruption touches only ∼16% of length-split predictions, and the remaining deficit is diffuse (right relationships, right values, subtly wrong assembly).

7 Discussion

One property, three predictions. The results cohere around a single structural property of the output space. Because a Cypher pattern has essentially one canonical surface form, sampling produces no useful diversity (P1: string voting falls below greedy) and there is little for a reasoning process to explore or for a selector to choose among; because that form is a globally coupled connected object, decomposing it fragments it (P3: causally shown in §6); and because execution results abstract away from surface form entirely, execution-grounded selection is the transfer that survives (P2). The same property explains all three outcomes—which is what makes it a hypothesis rather than a post-hoc grouping.

Why the SQL literature need not contradict this. Our negative result coexists with the SQL evidence once that evidence is read closely. The cleanest same-data CoT-vs-direct comparison in STaR-SQL is +6.4pp, not the headline +18pp (which includes best-of-16 verification); other celebrated results bundle CoT with RL or DPO, or prompt frontier models rather than fine-tune small ones. So the cross-formalism gap to explain is roughly +6pp (SQL) versus −2 to −24pp (Cypher, depending on regime)—and the hypothesis supplies structural reasons: decomposition matches SQL's compositional syntax but fragments Cypher's connected patterns; SQL's many equivalent forms give sampling-and-selection something to select over; and SQL benchmarks often require inferring literal values where ZOGRASCOPE hands them over, converting reasoning from an asset into an accumulation risk. A further, sharper possibility is that CoT's benefit shrinks as the direct baseline strengthens: our completion-only recipe produces an unusually strong direct model, compressing the headroom reasoning could occupy. Under this *baseline-strength* account, some published CoT gains may partly measure the weakness of their direct-answer controls. Testing whether CoT helps SQL *in this same matched pipeline* under execution accuracy (the staged Spider control) is the experiment that would separate these accounts, and we defer any claim until it lands.

A published artifact is not a control. The earlier framing of this project reported +0.1227 GLEU from CoT. The comparison was against a *published* fine-tuned adapter run through our inference harness—a baseline differing from our CoT model in training pipeline and inference context, not just in training target. Training the matched direct-answer control reversed the sign of the headline (Table 2); a leaked cross-split evaluation had likewise manufactured a "length-generalization SOTA" (§5.2). Neither error was exotic: one borrowed a control, the other trusted instance-ID disjointness. Both are common practice. The lesson is uncomfortable and general: *deltas attributed to an intervention are only meaningful against a baseline trained in the same pipeline*, and split hygiene must be verified at the content level, not the ID level. We document the failure in detail precisely because the corrected study was only possible after making it.

8 Limitations

SQL measurement. Our SQL evidence stops at canonical-AST exact match, a lower bound on semantic equivalence; the stability of the CoT deficit across metric levels is suggestive but not conclusive, and the Spider execution-accuracy control remains pending. We accordingly claim nothing about whether CoT helps SQL in this pipeline. **Single teacher and scale.** All traces come from one teacher (GPT-oss-120B) in one format family, and both student families are 8–9B models; a different teacher, trace style, or scale could shift the deltas, though the holistic/enumeration ladder shows the effect is not an artifact of one trace format. **Benchmark leakage.** Neo4j 2024

contains 31.7% verbatim question leakage (adversarial in direction—leaked questions often carry different gold queries—which we exploit as evidence in §5.2); absolute numbers on this benchmark should be read with that in mind. **Live-database drift.** Neo4j execution evaluation runs against public demo databases whose state changes over time (reference-failure counts varied from 166 to 51 between our evaluation dates); within-run comparisons are unaffected, but cross-run execution numbers carry a small same-denominator caveat. **Uncertainty.** The central deltas carry paired bootstrap 95% CIs, all excluding zero (Tables 1 and 3; SQL in §5.3); the Llama translation CIs and the SPARQL CI are pending retrieval of prediction files from the training cluster. Each arm is a single training run, so seed-to-seed training variance is not measured; the effect's directional consistency across seven independent comparisons, two model families, and three formalisms is the practical guard against a single-seed artifact. **Task character.** ZOGRASCOPE supplies entity values in the question, making it a copy-and-place task; benchmarks requiring value inference may dilute (or reverse) the error-accumulation component of the mechanism, though the fragmentation component is task-independent.

9 Conclusion

Which Text-to-SQL techniques transfer to Text-to-Cypher? Under matched-pipeline controls, two transfer: execution-grounded selection (result-clustering MBR beats both greedy decoding and string voting) and a careful SFT recipe (completion-only loss masking on schema-heavy prompts yields a direct-answer model far above the published baseline). Chain-of-thought distillation—the highest-profile candidate—does not: it modestly degrades Text-to-Cypher across two model families, across naive and execution-verified traces, and across in-distribution, compositional, and length-generalization regimes; string-diversity self-consistency fails alongside it. The constrained-output-space hypothesis predicts this pattern and the mechanism analysis explains it: Cypher's connected graph patterns are globally coupled objects with near-canonical surface forms, so decomposition fragments them (causally shown), reasoning-first generation accumulates copy errors, and sampling yields no diversity worth voting over—while execution results, which bypass surface form entirely, remain a useful signal. The broader lesson is methodological: the apparent CoT gain that motivated this project was an artifact of borrowing a published model as a control, and it took a matched in-pipeline baseline to find the sign of the true effect. Toolboxes do not transfer by default; controls do not transfer at all.

References

Vashu Chauhan et al. Mind the query: Large language models for cypher query generation. In *EMNLP 2025 Industry Track*, 2025.

CypherBench Authors. Cypherbench: Towards precise retrieval over full-scale modern knowledge graphs in the llm era. *ACL 2025*, 2025. arXiv:2412.18702.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. Qlora: Efficient finetuning of quantized llms. In *NeurIPS 2023*, 2023. arXiv:2305.14314.

G Feng, G Zhu, S Shi, Y Sun, Z Fan, S Gao, and J Hu. Robust nl-to-cypher translation for kbqa: Harnessing large language model with chain of prompts. In *CCKS 2023, Springer LNCS*, 2023.

Mingqian He, Yongliang Shen, Wenqi Zhang, Qiuying Peng, Jun Wang, and Weiming Lu. Star-sql: Self-taught reasoner for text-to-sql. In *ACL 2025*, 2025. arXiv:2502.13550.

Markus Hornsteiner et al. Real-time text-to-cypher query generation with large language models for graph databases. *Future Internet*, 16(12):438, 2024.

Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Lora: Low-rank adaptation of large language models. *arXiv preprint arXiv:2106.09685*, 2021.

M Kobeissi, N Assy, W Gaaloul, B Defude, and B Haidar. An intent-based natural language interface for querying process execution data. In *ICPM 2021*, 2021.

Xinyu Liu et al. A survey of nl2sql with large language models. *arXiv preprint arXiv:2408.05109*, 2024.

Zhaoyang Liu et al. Uncovering the impact of chain-of-thought reasoning for direct preference optimization. *ACL 2025*, 2025. arXiv:2502.11656.

Aidin Mohammadjafari et al. From natural language to sql: Review of llm-based text-to-sql systems. *arXiv preprint arXiv:2410.01066*, 2024.

Lunyiu Nie et al. Graphq ir: Unifying the semantic parsing of graph query languages with one intermediate representation. In *EMNLP 2022*, 2022.

Makbule Gulcin Ozsoy. Exploring iterative refinement for text2cypher. *Medium (Neo4j Developer Blog)*, 2025a.

Makbule Gulcin Ozsoy. Text2cypher: Data pruning using hard example selection. *arXiv preprint arXiv:2505.05122*, 2025b.

Makbule Gulcin Ozsoy. Enhancing text2cypher with schema filtering. *arXiv preprint arXiv:2505.05118*, 2025c.

Makbule Gulcin Ozsoy. Adapter fusion for multilingual text2cypher. *arXiv preprint arXiv:2601.16097*, 2026.

Makbule Gulcin Ozsoy, Leila Messallem, Jon Besga, and Gianandrea Minneci. Text2cypher: Bridging natural language and graph databases. *Proceedings of the GenAIK Workshop (COLING 2025)*, pages 100–108, 2025. arXiv:2412.10064.

Mohammadreza Pourreza and Davood Rafiei. Din-sql: Decomposed in-context learning of text-to-sql with self-correction. In *NeurIPS 2023*, 2023. arXiv:2304.11015.

Mohammadreza Pourreza and Davood Rafiei. Dts-sql: Decomposed text-to-sql with small large language models. *EMNLP 2024 Findings*, 2024. arXiv:2402.01117.

Mohammadreza Pourreza et al. Chase-sql: Multi-path reasoning and preference optimized candidate selection in text-to-sql. *ICLR 2025*, 2025a. arXiv:2410.01943.

Mohammadreza Pourreza et al. Reasoning-sql: Reinforcement learning with partial rewards for text-to-sql. *arXiv preprint arXiv:2503.23157*, 2025b.

Gaetano Rossiello et al. Rationalization models for text-to-sql. *ICLR 2025 Workshop on Reasoning and Planning for LLMs*, 2025. arXiv:2502.06759.

Zhihong Shao et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

STRuCT-LLM Authors. Struct-llm: Unifying tabular and graph reasoning with reinforcement learning for semantic parsing. *arXiv preprint arXiv:2506.21575*, 2025.

Chang-You Tai, Ziru Chen, Tianshu Zhang, Xiang Deng, and Huan Sun. Exploring chain of thought style prompting for text-to-sql. In *EMNLP 2023*, 2023. arXiv:2305.14215.

Parth Thaker and Guy Bresler. Struct-sql: Structured cot distillation. *arXiv preprint arXiv:2512.17053*, 2025.

Aman Tiwari, Shiva Krishna Reddy Malay, Vikas Yadav, Masoud Hashemi, and Sathwik Tejaswi Madhusudhan. Auto-cypher: Improving llms on cypher generation via llm-supervised generation-verification framework. In *NAACL 2025*, 2025. arXiv:2412.12612.

Quoc-Bao-Huy Tran, Aagha Abdul Waheed, Syed Mudasir, and Sun-Tae Chung. Refining text2cypher on small language model with reinforcement learning leveraging semantic information. *Applied Sciences*, 15(15):8206, 2025.

Bing Wang, Changyu Ren, Jian Yang, Xinnian Liang, Jiaqi Bai, Linzheng Chai, Zhao Yan, Qian-Wen Zhang, Di Yin, Xing Sun, and Zhoujun Li. MAC-SQL: A multi-agent collaborative framework for text-to-SQL. In *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025. arXiv:2312.11242.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, et al. Chain-of-thought prompting elicits reasoning in large language models. In *NeurIPS 2022*, 2022.

Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. Break it down: A question understanding benchmark. *TACL*, 2020. arXiv:2001.11770.

Tomer Wolfson et al. Weakly supervised text-to-sql parsing through question decomposition. *NAACL 2022*, 2022. arXiv:2112.06311.

Yonghui Wu, Mike Schuster, Zhifeng Chen, Quoc V Le, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

Chao Yang, Changyi Li, Xiaodu Hu, Hao Yu, and Jinzhi Lu. Enhancing knowledge graph interactions: A comprehensive text-to-cypher pipeline with large language models. *Information Processing and Management*, 63(1):104280, 2026.

Zhewei Yao et al. Arctic-text2sql-r1. *arXiv preprint arXiv:2505.20315*, 2025a.

Zhewei Yao et al. Excot: Iterative cot with dpo for text-to-sql. *ACL 2025 Findings*, 2025b. arXiv:2503.19988.

ZOGRASCOPE Authors. Zograscope: A benchmark for property graphs. *arXiv preprint arXiv:2503.05268*, 2025.